具体实施方式

以下结合附图与一个具体实施例（基于 TypeScript 实现，应用于一款竖屏像素风修仙 Roguelike 网页游戏）对本发明作进一步详细说明。应当理解，此处所描述的具体实施方式仅用于解释本发明，并不用于限定本发明的保护范围。

实施例 1：统一修饰向量定义

本发明将所有来源共用的修饰抽象为固定键集合的统一修饰向量 `Mods`，并定义十种攻击形态枚举 `AttackMode`：

```
// 所有来源共用的统一修饰（Effect DSL）
export interface Mods {
  atk:number; def:number; speed:number; crit:number; maxHp:number
  atkPct:number; defPct:number; hpPct:number
  aspd:number; lifesteal:number; skillPct:number; cdPct:number
}
export function emptyMods(): Mods {
  return { atk:0,def:0,speed:0,crit:0,maxHp:0,
           atkPct:0,defPct:0,hpPct:0,aspd:0,lifesteal:0,skillPct:0,cdPct:0 }
}
export type AttackMode = 'melee_arc'|'whirlwind'|'flying_sword'|'flying_sword_burst'
|'magic_bolt'|'frost_nova'|'frost_nova_burst'|'thunder_chain'|'sword_array'|'giant_sword'
```

实施例 2：多来源归一化聚合

本发明通过单一聚合函数 `aggregateMods`，将装备基座（含强化）与词缀、套装阶梯效果、心法（按功法等级缩放）、天赋等所有来源经同一 `addMod` 累加函数写入 `Mods` 向量：

```
// 汇总所有来源的修饰为统一向量
export function aggregateMods(s: GameState): Mods {
  const mods = emptyMods()
  // 装备基座（含强化）+ 词缀
  for (const key of Object.keys(s.equipment) as SlotKey[]) {
    const it = s.equipment[key]; if (!it) continue
    const mult = enhMult(it)
    addMod(mods,'atk',Math.round(it.atk*mult))
    addMod(mods,'def',Math.round(it.def*mult))
    addMod(mods,'speed',Math.round(it.speed*mult))
    addMod(mods,'crit', it.crit + 0.005*(it.enh??0))
```

```
      addMod(mods,'maxHp',Math.round(it.maxHp*mult))
      for (const af of it.affixes) addMod(mods, af.stat, af.value)
    }
    // 套装阶梯效果
    for (const setId of equippedSetIds(s)) {
      const count = setPieceCount(s, setId)
      for (const b of SETS[setId].bonuses)
        if (count>=b.count && b.mods)
          for (const [k,v] of Object.entries(b.mods)) addMod(mods, k as ModKey, v
as number)
    }
    // 心法（按功法等级缩放）
    for (const id of s.skillSlots.minds) {
      if (!id) continue
      const m = mindModsAt(id, s.skills[id]??1)
      for (const [k,v] of Object.entries(m)) addMod(mods, k as ModKey, v as number)
    }
    // 天赋
    const tm = talentMods(s)
    for (const [k,v] of Object.entries(tm)) addMod(mods, k as ModKey, v as number)
    return mods
}
```

由于所有来源共用同一向量与同一累加函数，新增属性来源时只需在其对应分支调用 `addMod`，无需为来源编写独立的合并逻辑。

实施例 3：攻击形态优先级链求值

本发明依据优先级链实时求值当前攻击形态：套装改写最高优先，其次为主修招式（需职业匹配），最后回退至职业默认形态；套装同时可改写神通形态：

```
// 攻击形态解析：套装 4 件改写 > 主修招式(需职业匹配) > 职业默认
export function resolveAttackMode(s: GameState): AttackMode {
    for (const setId of equippedSetIds(s)) {                    // ① 套装最高优先
      const count = setPieceCount(s, setId)
      for (const b of SETS[setId].bonuses)
        if (b.attackMode && count>=b.count) return b.attackMode
    }
    if (styleMatchesClass(s)) {                                 // ② 主修招式
      const def = SKILLS.get(s.skillSlots.style!)
      if (def?.attackMode) return def.attackMode
    }
    return classDefaultMode(s)                                  // ③ 职业默认
```

```
}
// 套装同时改写神通形态（如凡人套 4 件 → 大庚剑阵）
export function resolveSetArtMode(s: GameState): ArtMode | null {
  for (const setId of equippedSetIds(s)) {
    const count = setPieceCount(s, setId)
    for (const b of SETS[setId].bonuses)
      if (b.artMode && count>=b.count) return b.artMode
  }
  return null
}
```

实施例 4：数据驱动的形态登记

攻击形态与套装/招式效果均以配置数据登记，新增攻击形态时只需在配置表或枚举中登记，无需改动主战斗流程：

```
// 套装 4 件效果可直接改写攻击形态（数据驱动，无需改主流程）
interface SetBonus { count:number; mods?:Partial<Mods>; attackMode?:AttackMode;
artMode?:ArtMode; desc:string }
interface SetDef { id:string; name:string; pieces:string[]; bonuses:SetBonus[] }
// 招式定义携带 attackMode，装配即生效
interface SkillDef { id:string; name:string; type:SkillType; class:SkillClass
  attackMode?:AttackMode; power?:number; cd?:number; artMode?:ArtMode;
mods?:Partial<Mods>; desc:string }
```

实施例 5：单键触发与形态承载

在竖屏单手操作场景下，玩家仅以单一攻击键触发攻击。该攻击键在运行时调用 `resolveAttackMode` 取得由优先级链求值得出的当前攻击形态，从而以单一指令承载全部 Build 形态；当玩家切换装备、套装、招式或天赋时，聚合结果与最终形态自动随之变化，无需任何操作键的重新布局。

上述五个实施例共同实现了权利要求 1 所限定的"定义—聚合—求值—同步—扩展"完整方法，并揭示了从属权利要求 2~5 所进一步限定的各项技术特征。